Contrato de Integración: Accounting (Tema 08)

Referencia para los demás equipos del TPI. Define qué le pueden pedir a Accounting, qué eventos tienen que mandarle y qué eventos van a recibir de vuelta. Es solo la parte visible desde afuera, la "vidriera", no la documentación interna del microservicio.

0. La idea en una frase

Accounting es como la caja de un comercio: no decide precios ni promociones, solo cobra, guarda y entrega el comprobante. Quien decide cuánto vale algo (un desafío, un ítem, una puja) es siempre otro equipo. Accounting valida ese número (que sea positivo, que la cuenta exista y esté activa), lo registra y avisa.
Ejemplo: Motor de Desafíos le dice "el alumno aprobó, ganó 100 monedas". Accounting no pregunta por qué son 100 y no 120: suma las 100, anota el movimiento y avisa a Notificaciones.

1. Reglas generales, iguales para todos

- **Lo sincrónico pasa por el API Gateway.** Nadie llama directo a Accounting ni lee su base. Es como un edificio con portero: siempre se entra por la puerta principal.
- **Lo asincrónico va por Kafka.** Regla simple: si necesitás la respuesta para seguir, es un endpoint REST. Si solo avisás que algo pasó, es un evento.
- **Base path:** todos los endpoints van bajo /api/accounting.
- **Todo es por curso.** Un alumno en tres cursos tiene tres cuentas separadas. Las monedas ganadas en un curso no se pueden gastar en otro (RF-REC-01).
- **Accounting nunca calcula montos.** Siempre recibe el valor ya resuelto por el equipo dueño de la regla.
- **Nada se borra.** Un movimiento asentado nunca se edita ni se elimina. Una corrección es un movimiento nuevo que compensa al anterior, como en un libro contable de tinta: no tachás la línea, anotás una de ajuste abajo.
- **IDs:** todos los identificadores (alumno, curso, orden, desafío, ítem, hold) viajan como UUID en formato texto. En los ejemplos aparecen abreviados (stu-8821, h-9f21) solo para que se lean mejor.

Fases

Según la fasificación oficial del PRD, no todo entra en el MVP. Cada parte de este contrato lleva su etiqueta:

- **[MVP]**: cuentas, monedas, vidas, créditos por desafío, historial, reservas de vida.
- **[Fase 2]**: equipamiento (ítems, equipar, escudo), compra directa, desafío de recuperación de vida, reversión de vidas, endpoints para Backoffice.
- **[Fase 3]**: subastas.

Igual conviene que el contrato completo quede acordado desde ahora: "más adelante" significa

que se implementa después, no que no se diseña.

2. Contrato REST (síncrono, vía Gateway)

Base: /api/accounting. Todos usan los headers que ya propaga el Gateway (identidad y rol). Cuando el endpoint dice me, el alumno se identifica por su propio token: no puede pasar el ID de otro.

2.1 Consulta de cuenta [MVP]

Endpoint	Quién	Para qué
GET /courses/{courseId}/accounts/me	Alumno	Su saldo y sus vidas en ese curso.
GET /courses/{courseId}/accounts/{studentId}	ADMIN, PROFESSOR	Lo mismo, de un alumno puntual. PROFESSOR solo si dicta ese curso (ver sección 5).

```
{
  "availableBalance": 1100.00,
  "totalBalance": 1200.00,
  "reservedBalance": 100.00,
  "currentLives": 2,
  "reservedLives": 1,
  "totalLivesLost": 3,
  "isActive": true
}
```

availableBalance = totalBalance menos reservedBalance. Ejemplo: tenés 1200 monedas, pero 100 están "congeladas" en una compra en curso, así que podés gastar 1100.
Cuenta inexistente: 404. Cuenta recién matriculada que todavía se está creando: 404 con código ACCOUNT_NOT_YET_PROVISIONED.

2.2 Reservas de monedas activas [Fase 2/3]

Endpoint	Quién
GET /courses/{courseId}/students/me/coin-holds	Alumno
GET /courses/{courseId}/students/{studentId}/coin-holds	ADMIN

```
{ "activeHolds": [ { "holdId": "h-9f21", "orderId": "ord-1123",
  "orderType": "AUCTION_BID", "amount": 1800.00,
```

"expiresAt": "2026-09-06T15:00:00Z" }] }

2.3 Historial de movimientos, paginado [MVP]

Endpoint	Quién
GET /courses/{courseId}/accounts/me/coins-movements?page=&size=&from=&to=	Alumno
GET /courses/{courseId}/accounts/{studentId}/coins-movements?...	ADMIN
GET /courses/{courseId}/accounts/me/lives-movements?...	Alumno
GET /courses/{courseId}/accounts/{studentId}/lives-movements?...	ADMIN

Orden fijo, del más nuevo al más viejo; el cliente no lo puede cambiar.


```
{
  "content": [ { "id": "le-3301", "amount": 100.00, "balanceAfter":
1100.00,
    "movementType": "CHALLENGE_CREDIT", "sourceReferenceId":
"chl-450",
    "description": null, "createdDatetime": "2026-09-19T20:22:00Z",
    "reversionOfId": null, "reversedByEntryId": null } ],
  "page": 0, "size": 20, "totalElements": 47, "totalPages": 3
}
```


Si un movimiento fue corregido, aparecen los dos: el original (con reversedByEntryId) y el de ajuste (con reversionOfId).

2.4 Inventario de ítems [Fase 2]

Endpoint	Quién
GET /courses/{courseId}/accounts/me/items	Alumno
GET /courses/{courseId}/accounts/{studentId}/items	ADMIN
GET /courses/{courseId}/accounts/{studentId}/items-history	Solo ADMIN

{ "items": [{ "itemInstanceId": "item-901", "itemName": "Escudo",

```
  "itemType": "SHIELD", "status": "EQUIPPED",
  "createdDatetime": "2026-09-10T09:00:00Z" } ] }
```

Estados posibles de un ítem: AVAILABLE (lo tenés guardado), EQUIPPED (lo elegiste para el próximo desafío), RESERVED (está en uso en un desafío en curso), CONSUMED (ya se usó).

2.5 Endpoints para Roadmap (Tema 10)

Endpoint	Para qué
GET /courses/{courseId}/accounts/{studentId}/equip-summary [Fase 2]	Vidas e ítems para la pantalla de Roadmap, antes de arrancar un desafío. Se llama una vez por alumno.
GET /courses/{courseId}/accounts [MVP]	Todos los alumnos del curso, solo totalLivesLost y neverLostALife. Es el dato para desempatar el ranking; el ranking lo arma Roadmap.
GET /courses/{courseId}/accounts/{studentId}/public-detail [MVP]	Cuando un alumno toca la fila de otro en el ranking (RF-RNK-07). Accounting aporta monedas y vidas; XP e insignias las pone Roadmap. A propósito no devuelve el studentId: el detalle es anónimo.

```
equip-summary: { "currentLives": 2, "reservedLives": 0,
  "items": [ { "itemInstanceId": "item-901", "itemName": "Escudo",
    "itemType": "SHIELD", "status": "EQUIPPED" } ] }

accounts (ranking): [ { "studentId": "stu-8821", "totalLivesLost": 0,
"neverLostALife": true } ]

public-detail: { "totalBalance": 850.00, "currentLives": 2,
"totalLivesLost": 1 }
```

2.6 Endpoints de contingencia (Mercado y Motor)

No reemplazan a los eventos. Son como llamar al banco para confirmar una transferencia cuando la notificación no te llegó: se usan solo si un servicio sospecha que perdió un evento.

```
GET /holds/{holdId}          : estado de una reserva de monedas
GET /life-holds/{lifeHoldId} : estado de una reserva de vida
```

3. Contrato de eventos (Kafka)

Todo evento usa el envoltorio estándar de la plataforma: eventId (UUID), eventType, timestamp (ISO 8601 UTC), producer y payload. En los ejemplos solo se muestra el payload.

- **Idempotencia:** el eventId es la clave. Si Accounting recibe dos veces el mismo evento (reintento, entrega duplicada), la segunda vez no hace nada. Lo mismo esperamos de quienes consumen nuestros eventos.

- **Producer de Accounting:** siempre tema-08-accounting.
- **Pedido y respuesta:** cuando Accounting responde a un pedido, la respuesta lleva correlationId = el eventId del pedido original. Así el que pidió sabe a qué le están contestando, como el número de ticket de un reclamo.
- **Tópicos:** los tópicos de Accounting siguen el patrón accounting.<dominio>.commands (pedidos que le hacen) y accounting.<dominio>.events (hechos que publica). Los tópicos donde publican otros equipos son de ellos: los nombres de acá son propuestas a confirmar (ver sección 6).

3.1 Eventos que Accounting ESCUCHA

De Tema 02: Cursos y Matrícula [MVP]

La única forma de que nazca una cuenta. No existe endpoint para crear cuentas a pedido.

```
STUDENT_ENROLLED: crea la cuenta, o la reactiva si el alumno se
rematricula
{ "enrollmentId": "enr-987", "courseCohortId": "crs-14", "studentId":
"stu-8821",
  "validationType": "ROSTER_VALIDATED", "enrolledAt":
"2026-09-13T23:35:00Z" }

STUDENT_UNENROLLED: baja lógica de esa cuenta (reversible)
{ "enrollmentId": "enr-987", "courseCohortId": "crs-14", "studentId":
"stu-8821",
  "reason": "Baja solicitada por el alumno", "performedByUserId":
"usr-84",
  "unenrolledAt": "..." }

COURSE_COHORT_ARCHIVED: baja lógica de todas las cuentas del curso
(definitiva)
{ "courseCohortId": "crs-14", "courseBaseId": "cb-20", "archivedAt":
"..." }
```

Una baja no borra ni cambia el saldo, las vidas ni los ítems: la cuenta queda congelada y visible en modo lectura. Lo que sí hace es liberar todas las reservas pendientes (ver HOLD_RELEASED con ACCOUNT_DEACTIVATED en 3.2).
Nota de nombres: Cursos manda courseCohortId; en el resto del contrato ese mismo valor se llama courseId.

De Tema 12: Backoffice

```
PARAMETER_UPDATED [MVP]: vidas iniciales y máximas (PAR-12)
{ "parameterId": "PAR-12", "version": 3,
  "value": { "initialLives": 3, "maxLives": 3 } }

COIN_LEDGER_REVERSAL_REQUESTED [MVP]: un ADMIN pide corregir un
movimiento de monedas
{ "ledgerEntryId": "le-3250", "expectedAmount": 300.00,
  "description": "Compra duplicada por error de doble click",
```

```
  "actorId": "admin-42", "actorRole": "ADMIN" }
```
La descripción es obligatoria: es la justificación que queda en la auditoría. La reversión de vidas queda para Fase 2.

De Tema 03: Motor de Desafíos

Motor decide si el desafío se aprobó o no, y cuántas monedas vale. Accounting solo ejecuta. Ejemplo del ciclo completo: el alumno arranca un desafío, Motor pide "guardame 1 vida y su escudo". Accounting los aparta (nadie más puede usarlos). Al terminar, Motor avisa el resultado y Accounting devuelve la vida, la descuenta o gasta el escudo según corresponda.

```
CHALLENGE_ATTEMPT_RESERVATION_REQUESTED [MVP]: al iniciar un intento,
reservar
1 vida y los ítems equipados (todo o nada)
{ "studentId": "stu-8821", "courseId": "crs-14", "challengeId":
"chl-450",
  "equippedItemIds": ["item-901", "item-902"], "ttlSeconds": 5400 }

CHALLENGE_APPROVED [MVP]: acredita monedas, libera la vida e ítems no
usados
{ "studentId": "stu-8821", "courseId": "crs-14", "challengeId":
"chl-450",
  "lifeHoldId": "lh-501", "lifeResolution": "CHALLENGE_APPROVED",
  "coinsAwarded": 100.00, "consumedItemIds": ["item-905"] }

CHALLENGE_DISAPPROVED [MVP]: descuenta la vida reservada y consume
ítems usados
{ "studentId": "stu-8821", "courseId": "crs-14", "challengeId":
"chl-450",
  "lifeHoldId": "lh-501", "consumedItemIds": ["item-901"] }

RECOVERY_CHALLENGE_APPROVED [Fase 2]: suma 1 vida por desafío de
recuperación
{ "studentId": "stu-8821", "courseId": "crs-14", "challengeId":
"chl-999" }
```

- coinsAwarded llega ya calculado. Accounting solo valida que sea mayor a 0.
- lifeResolution en CHALLENGE_APPROVED puede ser CHALLENGE_APPROVED (aprobó sin arriesgar la vida) o SHIELD_APPLIED (falló un intento, el escudo lo protegió y después aprobó). En ambos casos la vida vuelve sin costo.
- En CHALLENGE_DISAPPROVED no hace falta mandar lifeResolution: la vida se pierde siempre.
- consumedItemIds lista los ítems que se gastaron. Los reservados que no aparecen ahí vuelven a estar disponibles.
- El desafío de recuperación solo aplica si el alumno tiene 0 vidas, y no usa reserva.

De Tema 09: Mercado, pedidos de reserva [Fase 2/3]

Tópico: accounting.holds.commands. Mismo mecanismo para compra directa (Fase 2) y subastas (Fase 3).

Ejemplo con una subasta: el alumno puja 1500, Accounting le "congela" 1500 monedas. Sube la puja a 1800: se congelan 1800 en total (no 3300). Si gana, se cobran las 1800. Si pierde, se descongelan sin costo.

```
HOLD_CREATE_REQUESTED: congelar un monto
{ "studentId": "stu-8821", "courseId": "crs-14", "orderId":
"ord-1123",
  "orderType": "AUCTION_BID", "amount": 1500.00, "ttlSeconds": 1800 }
(orderType: DIRECT_PURCHASE o AUCTION_BID. ttlSeconds solo para
subastas;
en compra directa se omite y el vencimiento lo define Accounting)

HOLD_INCREASE_REQUESTED: subir una puja (manda el TOTAL nuevo, no la
diferencia)
{ "holdId": "h-9f21", "newTotalAmount": 1800.00 }
(tiene que ser estrictamente mayor que el monto actual)

HOLD_CONFIRM_REQUESTED: cobrar lo congelado
{ "holdId": "h-9f21" }

HOLD_RELEASE_REQUESTED: descongelar, con motivo obligatorio
{ "holdId": "h-9f21", "releaseReason": "AUCTION_LOST" }
(Mercado puede mandar: AUCTION_LOST, AUCTION_CANCELLED,
PURCHASE_NOT_COMPLETED)
```

De Tema 09: Mercado, acreditaciones [Fase 2]

Una vez que Mercado ya cobró (con HOLD_CONFIRMED), avisa qué hay que entregarle al alumno.

```
LIFE_PURCHASE_CONFIRMED: acreditar vidas compradas
{ "studentId": "stu-8821", "courseId": "crs-14", "orderId":
"ord-2201", "quantity": 1 }
(si el alumno ya tiene el máximo de vidas, la compra se asienta igual
pero suma 0)

ITEM_CONFIRMED: acreditar un ítem comprado o ganado en subasta
{ "studentId": "stu-8821", "courseId": "crs-14", "orderId":
"ord-2305",
  "catalogItemId": "cat-item-42", "itemName": "Escudo", "itemType":
"SHIELD",
  "effect": "ABSORB_FAILURE" }
```

De Tema 10: Roadmap [Fase 2]

```
ITEM_EQUIP_REQUESTED: el alumno elige qué ítems llevar al próximo
desafío
{ "studentId": "stu-8821", "courseId": "crs-14",
  "itemInstanceIds": ["item-901", "item-902"] }
```

Cada ítem se evalúa por separado: si uno falla, los demás se equipan igual. Desequipar está

previsto pero su evento todavía no está definido (ver sección 6).

3.2 Eventos que Accounting PUBLICA

Movimientos de saldo (tópico accounting.ledger.events)

Hechos consumados, pensados para Notificaciones (Tema 11) y cualquier equipo que quiera enterarse. El campo movementType dice el origen exacto.

```
BALANCE_CREDITED [MVP]: entraron monedas
{ "studentId": "stu-8821", "courseId": "crs-14", "amount": 100.00,
  "totalBalance": 1100.00, "movementType": "CHALLENGE_CREDIT",
  "sourceReferenceId": "chl-450" }

BALANCE_DEBITED [Fase 2/3]: salieron monedas por una compra o subasta
ganada
(movementType: DIRECT_PURCHASE_DEBIT o AUCTION_WIN_DEBIT,
sourceReferenceId = orderId)

MOVEMENT_REVERSED [MVP]: un ADMIN corrigió un movimiento de monedas
(movementType: COINS_REVERSAL)

LIFE_CREDITED [MVP]: entró una vida
{ "studentId": "stu-8821", "courseId": "crs-14", "livesAwarded": 1,
  "currentLives": 1, "movementType": "LIFE_PURCHASE_CREDIT",
  "sourceReferenceId": "ord-2201" }
(movementType: INITIAL_CREDIT, LIFE_PURCHASE_CREDIT o
CHALLENGE_RECOVERY_CREDIT)

LIFE_DEBITED [MVP]: el alumno perdió una vida en un desafío
(movementType: CHALLENGE_DEDUCT, sourceReferenceId = challengeId)

ITEM_CREDITED [Fase 2]: el alumno recibió un ítem
{ "studentId": "stu-8821", "courseId": "crs-14", "itemInstanceId":
"item-1102",
  "itemName": "Escudo", "itemType": "SHIELD", "sourceReferenceId":
"ord-2305" }

ITEM_CONSUMED [Fase 2]: se gastaron ítems en un desafío
{ "studentId": "stu-8821", "courseId": "crs-14", "challengeId":
"chl-450",
  "consumedItems": [ { "itemInstanceId": "item-901", "itemName":
"Escudo" } ] }
```

Para Mercado: respuestas de reserva (tópico accounting.holds.events) [Fase 2/3]

Pedido	Si sale bien	Si sale mal
HOLD_CREATE_REQUESTED	HOLD_CREATED	HOLD_REJECTED
HOLD_INCREASE_REQUESTED	HOLD_INCREASED	HOLD_REJECTED
HOLD_CONFIRM_REQUESTED	HOLD_CONFIRMED	HOLD_REJECTED
HOLD_RELEASE_REQUESTED	HOLD_RELEASED	HOLD_REJECTED

```
HOLD_CREATED
{ "correlationId": "(eventId del HOLD_CREATE_REQUESTED)", "holdId":
"h-9f21",
  "studentId": "stu-8821", "courseId": "crs-14", "orderId":
"ord-1123",
  "orderType": "AUCTION_BID", "amount": 1500.00, "status": "PENDING",
  "expiresAt": "2026-09-06T15:00:00Z" }

HOLD_REJECTED
{ "correlationId": "(eventId del pedido)", "holdId": "h-9f21",
  "reason": "INVALID_HOLD_STATE", "message": "The hold is no longer in
the PENDING status" }
(reason: ACCOUNT_NOT_FOUND, HOLD_ALREADY_EXISTS, INSUFFICIENT_BALANCE,
INVALID_HOLD_STATE, HOLD_NOT_FOUND)

HOLD_RELEASED
{ "correlationId": "...", "holdId": "h-9f21", "studentId": "stu-8821",
  "courseId": "crs-14", "orderId": "ord-1123", "orderType":
"AUCTION_BID",
  "amount": 1800.00, "status": "RELEASED", "releaseReason":
"AUCTION_LOST",
  "releasedAt": "2026-09-06T14:33:02Z" }
```

Dos eventos que Mercado recibe sin haberlos pedido (importante para no romper el flujo):

- HOLD_EXPIRED: la reserva venció porque nadie la confirmó ni liberó a tiempo. No trae correlationId, porque nadie la pidió: la dispara un proceso automático de Accounting que revisa vencimientos cada ~45 segundos. Es como un estacionamiento medido: si no renovás, se libera solo.

  ```
  { "holdId": "h-9f21", "studentId": "stu-8821", "courseId":
  "crs-14",
    "orderId": "ord-1123", "orderType": "AUCTION_BID", "amount":
  1800.00,
    "status": "RELEASED", "releaseReason": "TTL_EXPIRED",
    "expiresAt": "2026-09-06T15:00:00Z", "expiredAt":
  "2026-09-06T15:00:38Z" }
  ```

- HOLD_RELEASED con releaseReason: ACCOUNT_DEACTIVATED: el alumno se dio de

baja o se archivó el curso mientras tenía una reserva. Si era la puja más alta de una subasta abierta, Mercado decide qué hacer (por ejemplo, recalcular el líder).

Para Motor de Desafíos: respuestas de reserva y liquidación [MVP]

Tópico: accounting.holds.events (ver sección 6).

```
CHALLENGE_ATTEMPT_RESERVATION_CREATED
{ "correlationId": "(eventId del REQUESTED)", "lifeHoldId": "lh-501",
  "studentId": "stu-8821", "courseId": "crs-14", "challengeId":
"chl-450",
  "expiresAt": "2026-09-19T21:45:00Z",
  "reservedItems": [ { "itemInstanceId": "item-901", "effect":
"ABSORB_FAILURE" } ] }

CHALLENGE_ATTEMPT_RESERVATION_REJECTED
{ "correlationId": "...", "studentId": "stu-8821", "courseId":
"crs-14",
  "challengeId": "chl-450", "reason": "INSUFFICIENT_LIVES" }
(reason: ACCOUNT_NOT_FOUND, ACCOUNT_INACTIVE, INSUFFICIENT_LIVES,
ITEM_NOT_AVAILABLE)

LIQUIDATION_REJECTED: Accounting no pudo procesar un
CHALLENGE_APPROVED/DISAPPROVED
{ "correlationId": "...", "studentId": "stu-8821", "courseId":
"crs-14",
  "challengeId": "chl-450", "lifeHoldId": "lh-501", "reason":
"LIFE_HOLD_INVALID_STATE" }
(reason: LIFE_HOLD_NOT_FOUND, LIFE_HOLD_INVALID_STATE,
ACCOUNT_NOT_FOUND, ACCOUNT_INACTIVE)
```

Ejemplo de rechazo: el alumno quiere arrancar un desafío pero tiene 1 vida y ya está reservada en otro desafío en curso. No hay vida libre, así que la respuesta es INSUFFICIENT_LIVES y el desafío no arranca.

Para Roadmap [Fase 2]

```
ITEM_EQUIP_RESULT: respuesta a ITEM_EQUIP_REQUESTED
{ "correlationId": "...", "studentId": "stu-8821", "courseId":
"crs-14",
  "equipped": ["item-901"],
  "rejected": [ { "itemInstanceId": "item-902", "reason":
"ITEM_NOT_AVAILABLE" } ] }
(reason: ITEM_NOT_AVAILABLE, ITEM_NOT_OWNED_BY_STUDENT)
```

4. Resumen de tópicos

Tópico	Quién publica	Quién escucha	Estado
accounting.holds.commands	Mercado	Accounting	Definido
accounting.holds.events	Accounting	Mercado, Motor, Roadmap	Definido
accounting.ledger.events	Accounting	Notificaciones y quien quiera	Definido
Eventos de matrícula y archivado	Cursos	Accounting	A confirmar con Tema 02
Reservas y resultados de desafío	Motor	Accounting	A confirmar con Tema 03
Compras confirmadas	Mercado	Accounting	A confirmar con Tema 09
Pedido de equipar ítems	Roadmap	Accounting	A confirmar con Tema 10
Parámetros y reversiones	Backoffice	Accounting	A confirmar con Tema 12

5. Lo que Accounting necesita de otros equipos

- **Cursos (Tema 02):** un endpoint de consulta que, dado un courseId y un profesor, responda si ese profesor dicta ese curso (con un sí/no alcanza). El Gateway confirma que alguien es profesor, pero no de qué curso. Sin esto, un profesor podría ver cuentas de cursos ajenos.
- **Identity & Users (Tema 01):** agregar accounting-service a la allowlist del Gateway. Sin eso, el Gateway responde 404 aunque el servicio esté registrado.
- **Backoffice (Tema 12):** publicar PAR-12 al arrancar la plataforma, para que las cuentas nuevas tengan un valor inicial de vidas.

6. Pendientes a acordar en la sesión de integración

- **Nombres de tópicos de otros equipos.** El estándar Kafka de la plataforma usa nombres como cursos.ciclo-vida (Tema 02) y desafios.resultados (Tema 03). Nuestro modelo asumía otros (course-events, challenges.reservations, challenges.resolutions, market.orders.events). Cada equipo emisor es dueño de su tópico: nos adaptamos al nombre que definan.
- **Valor de producer de cada equipo.** Hace falta una lista única (ej. tema-02-cursos, tema-03-desafios, tema-09-mercado) para no mezclar formatos.
- **Desequipar ítems.** Falta definir el evento con Roadmap (el inverso de ITEM_EQUIP_REQUESTED).
- **Cancelación de subasta.** Confirmar con Mercado si manda un

HOLD_RELEASE_REQUESTED por cada puja con AUCTION_CANCELLED, o si hace falta un pedido único por orden.

- **Tópico de respuestas a Motor y Roadmap.** Se unifica en accounting.holds.events; si Motor o Roadmap prefieren un tópico separado, se ajusta.
- **Techo de vidas para Roadmap.** Roadmap lo obtiene directo de Backoffice (evento o endpoint propio de ese equipo), no de Accounting. Falta que Roadmap y Backoffice acuerden el mecanismo.

7. Qué NO hace Accounting (para no pedirle de más)

- No calcula cuánto vale un desafío, un ítem ni una puja.
- No define el catálogo de ítems ni las reglas de subasta: eso es de Mercado.
- No arma el ranking: solo expone las vidas perdidas para desempatar.
- No maneja XP ni insignias: eso es de Roadmap.
- No tiene endpoint para crear cuentas: la cuenta nace de la matriculación.
- No corrige saldos por su cuenta: solo un ADMIN puede pedir una reversión, y queda auditada.

Documento vivo: si un equipo necesita un dato o un evento que no está acá, se conversa en la sesión de integración antes de asumirlo por su cuenta.